Empowering Users with Algorithmic Control and Transparency in Tourism Recommender Systems: A DSA-Compliant Approach

DAVID GALLOB, Technical University of Munich, Germany

Traditional recommendation systems operate as "black boxes," offering high predictive accuracy but leaving users with little control over how items are ranked and minimal transparency regarding the underlying decision logic. In e-tourism, where accommodation choices involve significant financial and emotional commitment, this lack of agency can reduce system trust and user satisfaction. Furthermore, recent regulatory frameworks, such as the European Union's Digital Services Act (DSA), mandate that online platforms explain the primary parameters of their recommender systems and provide profiling-free alternatives. This paper presents a web-based, interactive tourism recommender system prototype that addresses these challenges by putting user control and mathematical explainability at the center of the user experience. The prototype incorporates two distinct algorithmic paradigms: a compensatory Weighted Multi-Attribute Utility Theory (MAUT) approach, controlled via interactive sliders for five key dimensions (price, location, stars, user rating, and transit access), and a non-compensatory Constraint-Guided Filtering approach. We detail the system design, the mathematical formulations of the algorithms, and the UI-based explainability mechanisms, which include global description banners and on-demand, KaTeX-rendered mathematical breakdowns. Finally, we analyze critical implementation challenges and mathematical edge cases—such as division-by-zero singularities and score overshooting—and present resolutions to ensure system stability. This work demonstrates a concrete, compliant, and user-centric architecture for transparent modern recommender systems.

Additional Key Words and Phrases: Recommender Systems, User Control, Multi-Attribute Utility Theory, Constraint Filtering, Digital Services Act, Explainable AI, E-Tourism

1 INTRODUCTION

Recommendation systems (RS) have become ubiquitous in the digital economy, guiding user decisions in domains ranging from e-commerce and entertainment to travel and hospitality [1]. Despite their success, modern recommender systems are frequently critiqued for operating as "black boxes" [2]. Complex algorithms, such as deep neural networks or collaborative filtering engines, process massive volumes of user data to generate suggestions, yet they rarely explain the underlying rationale to the end user. This lack of transparency can hinder trust, increase cognitive overhead, and lead to user frustration, particularly in high-involvement domains like tourism, where travel planning represents significant financial expenditure and emotional investment [8].

To address these shortcomings, research in Human-Computer Interaction (HCI) and recommender systems has increasingly focused on user control and explainability [6, 12]. Rather than relying solely on automated profiling, interactive recommender systems allow users to explicitly guide the recommendation process. By adjusting preference weights or setting hard boundaries, users can align the system's output with their immediate constraints, such as budget, location, or quality preferences.

In addition to user-centered design motivations, the demand for algorithmic transparency is increasingly driven by regulatory mandates. The European Union's Digital Services Act (DSA), which entered full force recently, represents a landmark regulatory effort to govern online intermediaries [3]. Under Article 27 of the DSA ("Recommender system transparency"), online platforms must set out in their terms and conditions, in a clear and plain language, the main parameters used in their recommender systems, as well as any options for the recipients of the service to modify or influence those main parameters. Furthermore, Article 38 ("Profiling options") mandates that very large online platforms must provide at least one option for their recommender

Author's address: David Gallob, david.gallob@tum.de, Technical University of Munich, School of Computation, Information and Technology, Munich, Germany.

2026. ACM 2476-1249/2026/6-ART1
https://doi.org/

systems that is not based on user profiling. These regulations represent a paradigm shift, legally requiring developers to move away from pure black-box profiling and toward transparent, user-controlled algorithmic frameworks.

This paper presents the design, implementation, and evaluation of an interactive, DSA-compliant tourism recommender system prototype designed for hotel recommendations in Vienna and Munich. The system addresses the dual goals of transparency and user control by implementing two distinct algorithmic paradigms:

(1) **Algorithm A: Weighted Multi-Attribute Utility Theory (MAUT)**: A compensatory utility model where users utilize interactive sliders to assign weights to different hotel attributes (price, location, star rating, customer rating, and public transit access). The system normalizes these attributes, computes a weighted utility score, and ranks hotels accordingly, presenting a real-time percentage match score.
(2) **Algorithm B: Constraint-Based Filtering**: A non-compensatory filtering model where users define hard threshold limits for each attribute (e.g., maximum price or minimum rating). The system screens the database and displays only the subset of hotels that satisfy all constraints simultaneously, offering a profiling-free alternative.

To fulfill the explainability requirements of the DSA, the prototype features a global explanation banner detailing the active algorithm and an on-demand, collapsible "Mathematical Breakdown" panel on each hotel recommendation card. Using KaTeX, the interface renders the exact mathematical formulas and normalization equations used in the calculations, allowing users to trace how their inputs directly translate into the presented results.

The remainder of this paper is structured as follows. Section 2 reviews related work in tourism recommenders, multi-attribute utility theory, and explanation interfaces. Section 3 describes the system architecture and the dataset. Section 4 presents the mathematical formulations of the two algorithmic approaches. Section 5 details the user interface design and DSA compliance features. Section 6 analyzes critical mathematical edge cases and implementation bugs discovered during development, outlining their resolutions. Section 7 proposes a protocol for a future user study and outlines extensions, and Section 8 concludes the paper. The Appendix contains the complete database used in the study, followed by the author contribution table and GenAI disclosures.

2 RELATED WORK

The development of recommender systems in tourism is characterized by unique challenges that distinguish it from other application domains, such as movie or music recommendation [2]. Tourism decisions are high-involvement, occasional, and multi-attribute in nature [8]. Unlike low-cost consumer goods, booking a hotel or planning a vacation involves substantial financial risk, time commitment, and emotional expectation. Consequently, users are rarely satisfied with simple "top-N" recommendations; they require extensive information, comparison tools, and the ability to express specific, situational constraints.

Traditional tourism recommender systems (TRSs) are often categorized into collaborative filtering, content-based filtering, and hybrid approaches [4, 9]. While collaborative filtering leverages the historical ratings of similar users, it suffers from severe cold-start problems, as new hotels lack reviews, and occasional travelers lack historical profiles. Content-based systems match hotel features with user preference profiles, but they frequently struggle with explainability and cannot easily adapt to sudden shifts in user needs (e.g., a business traveler requiring a central location vs. a family seeking budget accommodations).

To overcome these limitations, multi-attribute decision-making frameworks, particularly Multi-Attribute Utility Theory (MAUT), have been applied to interactive recommendation [10]. MAUT provides a structured mathematical methodology for evaluating alternatives based on multiple, potentially conflicting criteria [5]. In a MAUT-based recommender, the system defines a utility function for each attribute (e.g., mapping price to a utility between 0 and 1) and aggregates these utilities using a weighted sum. Schmitt et al. [10] demonstrated that MAUT

is highly suited for conversational or interactive recommenders, as it maps directly to human decision-making processes. MAUT is fundamentally compensatory: a deficiency in one attribute (e.g., a high price) can be offset by an exceptionally high value in another (e.g., a prime location).

In contrast, constraint-based recommendation represents a non-compensatory approach. Instead of calculating a continuous score, constraint-based systems utilize explicit knowledge rules to filter out items that violate user-defined thresholds. This approach is highly predictable, transparent, and does not require historical user profiling, aligning perfectly with Article 38 of the DSA [3].

Prior research emphasizes that the success of interactive recommender systems depends heavily on their user interfaces [7, 11]. Providing explanation interfaces not only improves user understanding but also increases their trust and perceived control over the system [12]. Knijnenburg et al. [6] proposed a comprehensive framework showing that user control, when paired with high explainability, leads to higher satisfaction and system adoption. The integration of real-time sliders and mathematical breakdowns investigated in this work directly builds on these findings, operationalizing regulatory requirements into specific user interface patterns.

3 SYSTEM ARCHITECTURE AND IMPLEMENTATION

The prototype is designed as a client-side, responsive single-page application (SPA). The technical stack consists of React 18, Vite for the build system, and vanilla CSS for layout and styling. Client-side math rendering is powered by KaTeX, a fast LaTeX math rendering library, integrated via a custom React wrapper.

The system architecture is structured to support real-time user interaction. Figure 1 illustrates the flow of data through the system.

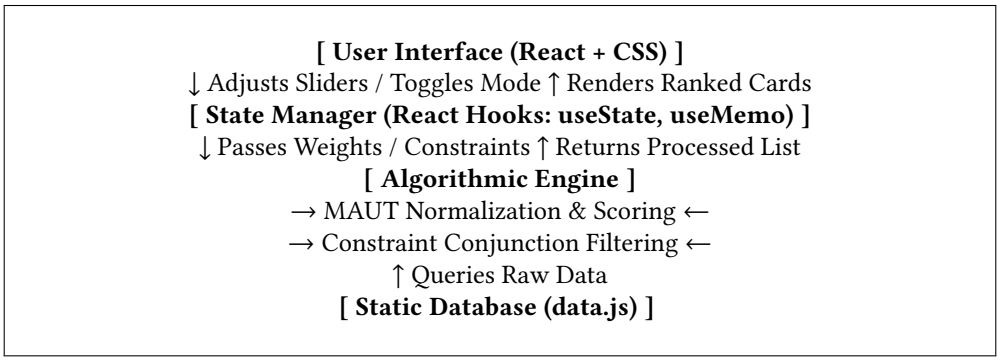

Fig. 1. Data Flow and Component Architecture of the Recommender Prototype.

The application state manages:

- The selected city ($c \in \{\text{Vienna}, \text{Munich}\}$).
- The active algorithmic mode ($m \in \{\text{Weighted MAUT}, \text{Constraint-Guided Filtering}\}$).
- The five attribute weights ($w_i \in [0, 100]$) for the MAUT mode.
- The five attribute bounds (b_i) for the Constraint Filtering mode.
- The expanded state of individual hotel cards (indicating whether the mathematical breakdown panel is visible).
- The global formula toggle (controlling whether plain-language explanations or mathematical LaTeX formulas are shown in the transparency banners).

The dataset, stored in a static module (‘data.js‘), comprises 20 hotels (10 in Vienna and 10 in Munich). The hotels were designed to represent a realistic and diverse spectrum of accommodations, ranging from budget-friendly

youth hostels to luxury five-star resorts. Each hotel X_j is defined by a unique identifier, name, city, description, and five key quantitative attributes:

(1) **Price per Night (p_j)**: Measured in USD ($). Ranging from $35 (Hostel Vienna West) to $550 (Mandarin Oriental Munich).
(2) **Distance to Center (d_j)**: Measured in kilometers (km) from the historical city center (Stephansplatz in Vienna, Marienplatz in Munich). Ranging from 0.1 km (Stephansplatz) to 28.0 km (Airport Express Inn Munich).
(3) **Star Rating (s_j)**: Ranging from 1 (basic tourist class) to 5 stars (luxury class).
(4) **Guest Review Score (r_j)**: A continuous rating between 1.0 (worst) and 10.0 (excellent), representing aggregated guest satisfaction.
(5) **Public Transport Access Score (t_j)**: A rating between 1.0 (poor) and 10.0 (excellent), representing proximity to subway stations, tram lines, and frequency of transit services.

By using ‘useMemo‘, the application recalculates recommendations in real time whenever the user modifies a slider or toggles a control. Because the calculations are performed client-side on a bounded dataset, updates occur instantly (< 16 ms), providing immediate visual feedback.

4 ALGORITHMIC FRAMEWORK

To support the user-controlled interaction model, we formalize the mathematical models behind the two algorithmic recommendation modes.

4.1 Algorithm A: Weighted Multi-Attribute Utility Theory (MAUT)

Weighted MAUT evaluates each hotel X_j in the active city set H_{city} by mapping its raw attribute values to normalized utility scores and computing a weighted average. The calculation proceeds in three stages:

4.1.1 Normalization of Attributes. Since attributes are measured in different units (USD, kilometers, star counts, and continuous scales), they must be normalized to a standard utility scale $u_i(x) \in [0, 1]$, where 0 represents the least desirable outcome and 1 represents the most desirable.

The attributes are categorized into two types: cost metrics (where lower values are better) and benefit metrics (where higher values are better). Furthermore, the system employs two different normalization boundaries:

- **Relative Normalization**: Normalizes values based on the maximum and minimum values present within the active city's dataset, maximizing the contrast between local options.
- **Absolute Normalization**: Normalizes values based on a fixed, theoretically defined range (e.g., 1 to 5 stars, or 1 to 10 rating scale).

The normalization functions for the five attributes are defined as follows:

(1) **Price (u_{price})**: A cost metric normalized relatively.

$$u_{\text{price}}(p_j) = \frac{p_{\max} - p_j}{p_{\max} - p_{\min}} \tag{1}$$

where $p_{\max} = \max_{k \in H_{\text{city}}} p_k$ and $p_{\min} = \min_{k \in H_{\text{city}}} p_k$.

(2) **Distance to Center (u_{location})**: A cost metric normalized relatively.

$$u_{\text{location}}(d_j) = \frac{d_{\max} - d_j}{d_{\max} - d_{\min}} \tag{2}$$

where $d_{\max} = \max_{k \in H_{\text{city}}} d_k$ and $d_{\min} = \min_{k \in H_{\text{city}}} d_k$.

(3) **Star Rating (u_{stars})**: A benefit metric normalized absolutely.

$$u_{\text{stars}}(s_j) = \frac{s_j - s_{\text{abs,min}}}{s_{\text{abs,max}} - s_{\text{abs,min}}} = \frac{s_j - 1}{4} \tag{3}$$

where $s_j \in [1, 5]$.

(4) **Guest Review Score (u_{rating})**: A benefit metric normalized absolutely.

$$u_{\text{rating}}(r_j) = \frac{r_j - r_{\text{abs,min}}}{r_{\text{abs,max}} - r_{\text{abs,min}}} = \frac{r_j - 1}{9} \tag{4}$$

where $r_j \in [1, 10]$.

(5) **Public Transport Score ($u_{\text{transport}}$)**: A benefit metric normalized absolutely.

$$u_{\text{transport}}(t_j) = \frac{t_j - t_{\text{abs,min}}}{t_{\text{abs,max}} - t_{\text{abs,min}}} = \frac{t_j - 1}{9} \tag{5}$$

where $t_j \in [1, 10]$.

4.1.2 Weighted Aggregation. Let $W = (w_{\text{price}}, w_{\text{location}}, w_{\text{stars}}, w_{\text{rating}}, w_{\text{transport}})$ be the vector of weights specified by the user, where each weight $w_i \in [0, 100]$. The total utility $U(X_j)$ for hotel X_j is calculated as:

$$U(X_j) = \frac{\sum_{i \in \text{Attrs}} w_i \cdot u_i(x_{ij})}{\sum_{i \in \text{Attrs}} w_i} \times 100\% \tag{6}$$

where Attrs = {price, location, stars, rating, transport} and x_{ij} represents the raw value of attribute i for hotel j. The resulting utility $U(X_j) \in [0, 100]\%$ represents the percentage match score. Hotels are sorted and presented to the user in descending order of $U(X_j)$.

4.2 Algorithm B: Constraint-Based Filtering

Constraint-Based Filtering does not calculate a numerical match score or rank hotels by preference. Instead, it screens the active city's dataset against five user-defined limits. Let C be the constraint tuple:

$$C = (P_{\text{max}}, D_{\text{max}}, S_{\text{min}}, R_{\text{min}}, T_{\text{min}}) \tag{7}$$

where:

- P_{max} is the maximum price per night ($P_{\text{max}} \in [\$30, \$600]$).
- D_{max} is the maximum distance from the center ($D_{\text{max}} \in [0.1, 30]$ km).
- S_{min} is the minimum star rating ($S_{\text{min}} \in [1, 5]$).
- R_{min} is the minimum guest rating ($R_{\text{min}} \in [1.0, 10.0]$).
- T_{min} is the minimum public transit score ($T_{\text{min}} \in [1.0, 10.0]$).

A hotel X_j is included in the recommended set R_{filtered} if and only if it satisfies the conjunction of all active constraints:

$$R_{\text{filtered}} = \{X_j \in H_{\text{city}} \mid f_{\text{filter}}(X_j) = \text{True}\} \tag{8}$$

where the filter function $f_{\text{filter}}(X_j)$ is defined as:

$$f_{\text{filter}}(X_j) = (p_j \leq P_{\text{max}}) \wedge (d_j \leq D_{\text{max}}) \wedge (s_j \geq S_{\text{min}}) \wedge (r_j \geq R_{\text{min}}) \wedge (t_j \geq T_{\text{min}}) \tag{9}$$

This formulation represents a non-compensatory filtering logic. A hotel that violates even a single constraint (such as exceeding the maximum price by $5) is immediately excluded from the results, regardless of whether it possesses perfect scores (e.g., 5 stars, 10.0 user rating) in all other categories.

4.3 Comparison of Decision Logic

The two models represent fundamentally different decision-making styles, as summarized in Table 1.

Table 1. Comparison of the Implemented Algorithmic Paradigms

Dimension	Algorithm A: Weighted MAUT	Algorithm B: Constraint Filtering
Decision Logic	Compensatory (trade-offs allowed)	Non-compensatory (hard boundaries)
Output Type	Ranked list with continuous match scores	Unordered subset of matching items
User Controls	Relative importance weights ($w_i \in [0, 100]$)	Threshold limits ($P_{\max}$, $D_{\max}$, etc.)
Cognitive Style	Optimizing	Screening / Satisficing
DSA Compliance	Explains weights	Explains inclusion/exclusion criteria
Profiling Dependency	None (Profiling-free, real-time input)	None (Profiling-free, real-time input)

5 USER INTERFACE DESIGN AND EXPLAINABILITY

The prototype interface was designed with a glassmorphism aesthetic (utilizing translucent backdrops, vibrant background gradients, and smooth animations) to create a premium, modern user experience. More importantly, the interface was structured to maximize explainability and user control.

5.1 DSA-Compliant Explanation Interfaces

To comply with the requirements of the Digital Services Act, transparency mechanisms are built directly into the primary interaction flow rather than being hidden in auxiliary pages.

5.1.1 Global Explanation Banners. At the top of the interface, a dedicated, colored information card dynamically adapts to the selected algorithm.

- In **Weighted MAUT** mode, the banner displays a blue theme, introducing the concept of multi-attribute weighting.
- In **Constraint-Guided** mode, the banner displays a green theme, introducing the logic of set reduction and non-compensatory filtering.

Both banners include a "Show Mathematical Details" toggle button. Clicking this button expands a sub-panel that renders the mathematical equations in LaTeX (Equation 6 for MAUT, and Equation 9 for Constraint-Guided Filtering) alongside a list of definitions. This design provides a tiered explanation model: novice users receive a brief, plain-language summary of how the parameters are aggregated, while expert users or regulators can inspect the exact mathematical formulas.

5.1.2 On-Demand Collapsible Math Breakdowns. For the Weighted MAUT mode, each hotel card features a "Show Mathematical Breakdown" button. Expanding this panel reveals a step-by-step trace of the utility calculation for that specific hotel. The panel displays:

- The general formula: Score = $\frac{\sum(w_i \cdot \text{norm}_i)}{\sum w_i}$
- A tabular breakdown for each of the five attributes, displaying the normalized utility score (u_i), the user-defined weight (w_i), and the product ($w_i \cdot u_i$).
- The summation step, showing the sum of products divided by the sum of weights.
- The final rounded match percentage.

For example, if a user selects Vienna and sets all weights to 50, and inspects the "Boutique Hotel am Stephansplatz" card, the breakdown displays:

- Price utility: $0.86 \times 50 = 43$
- Location utility: $1.00 \times 50 = 50$
- Stars utility: $0.75 \times 50 = 38$
- Rating utility: $0.91 \times 50 = 46$
- Transport utility: $1.00 \times 50 = 50$
- Sum of products: 227
- Total weight: 250
- Final Match: $\frac{227}{250} \times 100\% = 91\%$

This feature provides absolute transparency, eliminating the perception of the recommender system as a black box. The user can see exactly why a hotel is ranked first or second, and can directly trace the mathematical impact of adjusting a slider.

6 IMPLEMENTATION CHALLENGES & MATHEMATICAL EDGE CASES

During the development of the prototype, several critical mathematical bugs were identified, particularly within the MAUT calculation engine. These bugs are characteristic of interactive multi-attribute systems that rely on dynamic user inputs. This section details these challenges and their mathematical resolutions.

6.1 The Division-by-Zero Singularity on Zero Weights

6.1.1 The Problem. In early implementations of the Weighted MAUT algorithm, the denominator of the aggregation function was the sum of the user-defined weights, $\sum_{i=1}^{5} w_i$. In an interactive interface, users are free to move all sliders to 0. If a user set all weights to 0, the sum of weights became 0, leading to a division-by-zero error:

$$U(X_j) = \frac{\sum_{i=1}^{5} 0 \cdot u_i(x_{ij})}{0} = \frac{0}{0} \tag{10}$$

In JavaScript, this evaluation resulted in ‘NaN‘ (Not a Number), causing the hotel cards to fail to render, crashing the results grid, or rendering blank match badges.

6.1.2 The Resolution. To ensure mathematical robustness, a fallback mechanism was introduced. The total weight calculation in the codebase was modified using a logical OR operator: “‘javascript const totalWeight = weights.price + weights.location + weights.stars + weights.rating + weights.publicTransport || 1; “‘ If the sum of weights evaluates to 0, the code defaults the denominator to 1. This prevents the application from crashing. However, if the total weight is 1 and all individual weights are 0, the raw score sum remains 0, yielding a match percentage of:

$$U(X_j) = \frac{0}{1} \times 100\% = 0\% \tag{11}$$

To improve the user experience, a preferred scientific alternative is to treat the case where all weights are zero as an "equal weighting" scenario, where each attribute is automatically assigned an equal weight of 20 (resulting in a default equal aggregation). This is implemented as: “‘javascript const weightSum = weights.price + weights.location + weights.stars + weights.rating + weights.publicTransport; const totalWeight = weightSum === 0 ? 5 : weightSum; “‘ In this case, if all weights are zero, the algorithm defaults to:

$$U(X_j) = \frac{\sum_{i=1}^{5} 1 \cdot u_i(x_{ij})}{5} \times 100\% \tag{12}$$

which represents a standard, unweighted average utility, preserving the relative ranking of hotels even when all sliders are set to zero.

6.2 Relative Normalization Singularity (Zero-Range Denominator)

6.2.1 The Problem. Relative normalization maps raw values to a $[0, 1]$ range based on the local minimum and maximum values:

$$u_i(x_{ij}) = \frac{x_{\max} - x_{ij}}{x_{\max} - x_{\min}} \tag{13}$$

This formula assumes that $x_{\max} > x_{\min}$. However, if a city dataset contains only hotels with the exact same value for an attribute, or if the filtered subset in a hybrid mode reduces the candidate set to hotels with identical prices or distances, the range becomes zero ($x_{\max} - x_{\min} = 0$). In this scenario, evaluating the utility leads to division by zero:

$$u_i(x_{ij}) = \frac{x_{\max} - x_{ij}}{0} \tag{14}$$

which results in ‘Infinity‘ or ‘NaN‘ values, destabilizing the ranking.

6.2.2 The Resolution. The normalization engine was updated to include conditional checks. For price and location normalization, the code checks if the maximum and minimum values are equal: “‘javascript const normPrice = maxPrice === minPrice ? 1.0 : (maxPrice - hotel.price) / (maxPrice - minPrice); const normLoc = maxLoc === minLoc ? 1.0 : (maxLoc - hotel.location) / (maxLoc - minLoc); “‘ If $x_{\max} = x_{\min}$, the utility is set to 1.0 for all hotels. This mathematically represents the logic that if all hotels are identical in a given attribute, they all share the maximum possible utility (1.0) for that attribute, neutralizing its impact on the final ranking.

6.3 Match Score Overshooting and Precision Errors

6.3.1 The Problem. In early stages, developers noted that match scores occasionally overshot the 100% boundary (e.g., rendering scores of 101% or 105%). This occurred due to inconsistencies in the normalization boundaries of the benefit metrics. For instance, if absolute normalization was calculated as:

$$u_{\text{stars}}(s_j) = \frac{s_j - 1}{4} \tag{15}$$

but a hotel in the database was erroneously recorded with a star rating of 6 stars, the resulting utility would be 1.25, pushing the final match score over 100

6.3.2 The Resolution. The implementation was secured by applying strict range boundaries and precision clamping. All raw normalized scores are clamped to the interval $[0.0, 1.0]$:

$$u_i(x) = \max\left(0.0, \min\left(1.0, u_i(x)\right)\right) \tag{16}$$

Additionally, the final match percentage is calculated, clamped to $[0, 100]$, and rounded to the nearest integer: “‘javascript const percentageScore = Math.min(100, Math.max(0, Math.round((rawScore / totalWeight) * 100))); “‘ This ensures that the output is always an integer between 0% and 100%, preventing any visual anomalies in the user interface.

7 FUTURE WORK

While the current prototype demonstrates the feasibility of user-controlled, transparent tourism recommenders, several research and development paths remain open.

7.1 The Hybrid Algorithmic Approach

A natural evolution of the two modes is a hybrid algorithm that combines the strengths of both. Currently:

- Weighted MAUT is compensatory, which can lead to high cognitive load as users must constantly balance multiple sliders to filter out unacceptable options.

- Constraint-Based Filtering is non-compensatory and does not rank the matching items, which can leave the user with too many options or an empty set.

A hybrid recommender would operate as a two-stage pipeline:

(1) **Stage 1: Constraint Filtering (Non-Compensatory screening)**: The system applies user-defined constraints to filter out hotels that are completely unacceptable (e.g., price > $250, or distance > 10 km). This reduces the candidate pool to a set of feasible options.
(2) **Stage 2: Weighted MAUT (Compensatory ranking)**: The system applies the user's preference weights to rank only the hotels that passed Stage 1.

This hybrid approach represents a realistic model of human decision-making, where decision-makers first eliminate unacceptable alternatives and then optimize among the survivors. It maintains full transparency and complies with the DSA, while reducing user cognitive load.

7.2 Proposed User Evaluation Protocol

To evaluate the user experience (UX) and the effectiveness of the explainability features, we propose a controlled user study protocol.

7.2.1 Hypotheses. We formulate three primary hypotheses:

- **H1 (Perceived Control)**: Users interacting with the Weighted MAUT or Constraint Filtering interfaces report a higher sense of control over recommendations compared to a baseline "black-box" interface.
- **H2 (System Trust)**: Exposing the collapsible mathematical breakdown panels increases user trust in the system's recommendations without causing information overload. **H3 (DSA Compliance Impact)**: The presence of the global explanation banner and KaTeX formulas increases user comprehension of the algorithm's inner workings.

7.2.2 Experimental Design. We propose a within-subjects laboratory study with $N = 30$ participants. Each participant will perform travel planning tasks under three experimental conditions:

(1) **Condition 1 (Baseline - Black Box)**: A traditional recommender interface displaying hotel cards ranked by a hidden algorithm, with no sliders and no explanation panels.
(2) **Condition 2 (Weighted MAUT with On-Demand Explanations)**: The interactive slider-based interface with collapsible math breakdown panels.
(3) **Condition 3 (Constraint Filtering with Explanations)**: The constraint-based interface showing absolute threshold sliders.

The ordering of conditions will be counterbalanced to prevent learning effects. Following each condition, participants will complete standardized questionnaires to measure:

- **System Usability**: Measured via the System Usability Scale (SUS).
- **User Experience & Trust**: Measured using the ResQue (Recommender Systems Quality of Experience) framework, focusing on trust, perceived accuracy, control, and transparency.
- **Cognitive Load**: Measured via the NASA-TLX (Task Load Index) to evaluate if exposing mathematical details increases mental demand.

The results of this study will guide the refinement of the UI, helping to strike a balance between complete mathematical transparency and user cognitive ease.

8 CONCLUSION

The era of black-box recommender systems is facing both user-experience and regulatory challenges. This paper presented an interactive, transparent tourism recommender system prototype designed to address the

compliance requirements of the EU Digital Services Act (DSA). By implementing both a compensatory Weighted Multi-Attribute Utility Theory (MAUT) model and a non-compensatory Constraint-Based Filtering model, the prototype provides users with direct, profiling-free control over recommendation logic. We integrated tiered explainability features, combining simple plain-language summaries with on-demand, KaTeX-rendered mathematical breakdowns. Finally, we analyzed and resolved mathematical edge cases, including division-by-zero singularities and score overshooting, ensuring a stable and robust system. This architecture serves as a blueprint for developers seeking to combine rich user control, modern web aesthetics, and strict regulatory compliance in recommender system design.

ACKNOWLEDGMENTS

The author would like to thank the teaching staff of the Lab Course: Projects in Recommender Systems (SS 2026) at the Chair of Connected Mobility, Technical University of Munich, for their guidance and feedback throughout the project.

REFERENCES

[1] Gediminas Adomavicius and Alexander Tuzhilin. 2005. Toward the next generation of recommender systems: A survey of the state-of-the-art and possible extensions. *IEEE Transactions on Knowledge and Data Engineering* 17, 6 (2005), 734–749. https://doi.org/10.1109/TKDE.2005.99

[2] Joan Borras, Antonio Moreno, and Aida Valls. 2014. Intelligent tourism recommender systems: A survey. *Expert Systems with Applications* 41, 16 (2014), 7370–7389. https://doi.org/10.1016/j.eswa.2014.06.007

[3] European Parliament and Council of the European Union. 2022. Regulation (EU) 2022/2065 of the European Parliament and of the Council of 19 October 2022 on a Single Market For Digital Services and amending Directive 2000/31/EC (Digital Services Act). http://data.europa.eu/eli/reg/2022/2065/oj.

[4] Mohamed Elyes Ben Haj Kbaier, Hela Masri, and Saoussen Krichen. 2017. A Personalized Hybrid Tourism Recommender System. In *2017 IEEE/ACS 14th International Conference on Computer Systems and Applications (AICCSA)*. 244–250. https://doi.org/10.1109/AICCSA.2017.12

[5] Ralph L. Keeney and Howard Raiffa. 1993. *Decisions with Multiple Objectives: Preferences and Value Trade-Offs*. Cambridge University Press, Cambridge, UK.

[6] Bart P. Knijnenburg, M. C. Willemsen, Z. Gantner, H. Soncu, and C. Newell. 2012. Explaining the user experience of recommender systems. *User Modeling and User-Adapted Interaction* 22, 4-5 (2012), 441–504. https://doi.org/10.1007/s11257-011-9118-4

[7] Sean M. McNee, John Riedl, and Joseph A. Konstan. 2006. Being accurate is not enough: How accuracy metrics have hurt recommender systems. (2006), 1097–1101. https://doi.org/10.1145/1125451.1125659

[8] Francesco Ricci. 2011. Travel recommender systems. In *Recommender Systems Handbook*. Springer, 527–556. https://doi.org/10.1007/978-0-387-85820-3_17

[9] Joy Lal Sarkar, Abhishek Majumder, Chhabi Rani Panigrahi, Sudipta Roy, and Bibudhendu Pati. 2023. Tourism recommendation system: a survey and future research directions. *Multimedia Tools and Applications* 82, 6 (2023), 8983–9027. https://doi.org/10.1007/s11042-022-12167-w

[10] Christian Schmitt, Dietmar Dengler, Michael Bauer, et al. 2002. The MAUT-Machine: An Adaptive Recommender System. In *Proceedings of the ABIS Workshop* (Hannover, Germany).

[11] Kirsten Swearingen and Rashmi Sinha. 2001. Beyond algorithms: An HCI perspective on recommender systems. (2001).

[12] Nava Tintarev and Judith Masthoff. 2012. Evaluating recommendation explanations: A survey. *User Modeling and User-Adapted Interaction* 22, 4-5 (2012), 399–439. https://doi.org/10.1007/s11257-011-9106-8

A COMPLETE HOTEL DATABASE

Table 2 presents the complete dataset of 20 hotels used in the recommender system prototype, split between Vienna (v1–v10) and Munich (m1–m10).

B AUTHOR INFORMATION AND CONTRIBUTION DETAILS

In accordance with course requirements, Table 3 documents the academic details, contact endpoints, and contributions of the group member.

Table 2. Hotel Database Attributes

ID	Hotel Name	Price ($/night)	Distance (km)	Stars	Rating (/10)	Transport (/10)
		Vienna Hotels				
v1	Grand Imperial Vienna	350	0.5	5	9.6	9.5
v2	Boutique Hotel am Stephansplatz	180	0.1	4	9.2	10.0
v3	Danube Riverside Hotel	95	4.5	3	8.0	6.5
v4	Schönbrunn Palace Suites	290	6.0	5	9.8	8.0
v5	Hostel Vienna West	35	3.2	1	7.5	9.0
v6	Art Deco Hotel Prater	140	2.8	4	8.7	8.5
v7	Economy Inn Vienna	60	5.5	2	6.8	5.0
v8	Museum Quarter Retreat	210	1.2	4	9.4	9.0
v9	Green Oasis Hotel	110	7.5	3	8.9	4.0
v10	The Ritz-Carlton Vienna	450	0.8	5	9.7	10.0
		Munich Hotels				
m1	Bayerischer Hof	420	0.4	5	9.5	9.5
m2	Marienplatz Business Hotel	160	0.2	4	8.8	10.0
m3	Oktoberfest Lodge	85	2.5	2	7.2	8.0
m4	Englischer Garten Retreat	250	3.0	4	9.3	7.0
m5	Schwabing Youth Hostel	40	4.0	1	8.1	8.5
m6	Olympic Park Hotel	130	6.5	3	8.5	7.5
m7	Airport Express Inn	90	28.0	3	7.8	9.0
m8	Viktualienmarkt Boutique	190	0.6	4	9.1	9.5
m9	Isar River View	210	1.8	4	8.9	6.0
m10	Mandarin Oriental Munich	550	0.5	5	9.8	9.5

Table 3. Author Identification and Contribution Table

Full Name (as on transcript)	Email Address	GitHub Handle	Key Contributions
David Gallob	david.gallob@tum.de	Daev	Full design and implementation of the React application; formulation and integration of the Weighted MAUT algor formulation and coding of the Constraint-Filtering logic; implementation of KaTeX explanations and UI styling; resolution of mathematical singularities and edge cases; drafting of the final scientific paper and repository disclos

C GENERATIVE AI DISCLOSURE

This project utilized Generative AI tools (specifically Google DeepMind's Gemini 3.5 Flash) during development. The AI was employed as a pair-programming assistant to:

- Draft and optimize the user interface layouts and CSS styling rules.
- Debug mathematical singularities (such as relative normalization division by zero) and recommend JavaScript state handling best practices.

- Assist in structuring and editing the LaTeX code of this scientific report.

All design decisions, algorithmic formulations, bug resolutions, and report text were audited, verified, and approved by the human author to ensure compliance with the scientific standards of the Technical University of Munich.